[redacted]

Mark: 5/20 (total score: 5/20)

+196/1/60+

**ECA - Programmation fonctionnelle** **25/06/2024**

**Durée : 90 min. Le seul document autorisé est une feuille A4 blanche manuscrite**

Répondre aux différentes questions en écrivant **lisiblement** à l'intérieur du cadre. Ne pas cocher les cases qui sont dans les rectangles gris, elles sont dédiées au correcteur.

[redacted]

# 1 Système de fichier

On rappelle qu'un système de fichiers est constitué de répertoires contenant eux même des fichiers ainsi que d'autres répertoires (appelés sous-répertoires).

On considère le type suivant pour représenter les élements d'un système de fichiers :

```
type sf =
    | Repertoire of string * (sf list)
    | Fichier of string * int * string
```

Les données associées au constructeur `Repertoire` sont le nom du répertoire et la liste des fichiers et sous-répertoires qu'il contient. Les données associées au constructeur `Fichier` sont le nom du fichier, sa taille et sa *sorte*.

Par exemple, un fichier `README.txt` d'une taille de 856 octets de sorte `texte` serait représenté par `Fichier("README.txt", 856, "texte")`. Un fichier `maquette.jpg` d'une taille de 789456 octets de sorte `image` serait représenté par `Fichier("maquette.jpg", 789456, "image")`. Un répertoire nommé `monprojet` contenant uniquement les deux fichiers précédents serait représenté par `Repertoire("monprojet",[ Fichier("README.txt", 856, "texte"); Fichier("maquette.jpg", 789456, "image") ])`.

**Question 1** (0,5 pt) Donner le type de la fonction `taille` qui donne la taille totale occupée par les fichiers d'un système de fichiers.

F J

0.5/0.5

taille : sf → int

**Question 2** (2 pt) Donner le code de la fonction `taille` qui donne la taille totale occupée par les fichiers d'un système de fichiers.

F PJ J

1.5/2

```
def rec taille (f : 'sf) : int =
  match f with
  | Repertoire(_,[]) -> 0
  | Repertoire(_,Fichier(_,x,_)::l) -> x+ taille(~~Repert~~
                                        Repertoire(_,l) )

;;
```

**Question 3** (3 pt) On considère la fonction `fold_fichiers: ('a -> string -> int -> string -> 'a) -> 'a -> sf -> 'a`. Un appel à `fold_fichiers f init sysfich` va parcourir le système de fichiers `sysfich` en calculant au fur et à mesure pour chaque fichier une nouvelle valeur d'accumulateur. Cette nouvelle valeur est obtenue via `f acc n t d` pour un fichier `Fichier(n,t,d)` et une valeur précédente de l'accumulateur `acc`. La valeur de départ de cet accumulateur est `init`.

F PJ J

0/3

```
let fold_fichiers (
```

**Question 4** (1 pt) Donner le code d'une fonction `ajoute: string -> string list -> string list` qui ajoute un élement à une liste seulement s'il n'est pas déjà dans cette liste.

0/1

```
let rec ajoute (c: string) (l: string list) : string list =
  match l with
  |[] -> [c]
  |c1 :: l1 -> if c1 == c then l
               else ajoute c l1
;;
```

**Question 5** (1,5 pt) En utilisant, entre autres, la fonction `fold_fichiers`, donner le code de la fonction `sortes: sf -> string list` qui donne la liste des sortes de fichiers présentes dans son argument, sans doublon.

0/1.5

```
let sortes (s: sf) : string list =
  fold_fichers (fun f -> Fichier(_, _, type))
    [] sf
;;
```

**Question 6** (3 pt) On souhaite maintenant pouvoir filtrer le contenu d'un système de fichier. Un filtre de fichier est une fonction qui prend un nom, une taille et une sorte de fichier et renvoie un booléen. On dit qu'un fichier satisfait le filtre si le résultat du filtre appliqé au nom, à la taille et à la sorte du fichier renvoie `true`.
On considère la fonction `filtre_fichiers: (string -> int -> string -> bool) -> sf list -> sf list`. Le premier argument de cette fonction est un filtre de fichier. Le résultat de cette fonction est une liste de systèmes de fichiers dans lesquels on aura retiré tous les fichiers qui ne satisfont pas le filtre.

Donner le code de la fonction `filtre_fichiers`.

0/3

```
let filtre_fichers (f: string -> int -> string -> bool)
(l: sf list): sf list =
match l with
| |
```

## 2 Arbres d'expressions annotés

On considère le type suivant pour représenter des expressions :

```
type 'a expr =
    | Const of 'a * int
    | Plus of 'a * 'a expr * 'a expr
```

Ce type ressemble à des arbres d'expression comme vu en TP, mais contient en plus dans chaque nœud une valeur, que l'on appellera une *annotation*.

**Question 7** (1 pt) Donner le type et le code de la fonction `annotation_racine` qui renvoie l'annotation associée à la racine d'une expression.

0.5/1

```
annotation_racine : 'a expr → 'a
let rec annotation_racine ( a : 'a expr ) : 'a =
  match a with
  | Const ( x , _ ) → x
  | Plus ( y , x1 , _ ) → annotation_racine x1
  | Plus ( y , _ , x2 ) → annotation_racine x2
;;
```

**Question 8** (2 pt) Donner le type et le code de la fonction `annotation_eval` qui prend une expression et renvoie une expression identique à la première sauf pour les annotations qui seront remplacées par la valeur l'expression.

0.5/2

```
type annotation_eval 'a expr → 'a expr
let rec annotation_eval ( a : 'a expr ) : 'a expr =
  match a with
  | Const ( x , y ) → Const ( y , y )
  | Plus ( b , x1 , x2 ) →
      Plus ( b , annotation_eval x1 , annotation_eval x2 )
;;
```

**Question 9** (2 pt) On suppose que les expressions sont maintenant lues dans un fichier. Après la lecture, les expressions sont annotées par des paires `(debut,fin)` correspondant à la position de début et de fin de l'expression dans le fichier. Cependant, les positions pour les nœuds `Plus` correspondent au symbole + et pas à toute l'expression. Donner le type et le code d'une fonction `corrige_positions` qui change ces annotations de façon à ce que chaque expression soit annotée avec le début et la fin de toute l'expression.

0/2

```
type corrige_position : 'a expr → 'a expr
let rec corrige_position (a : 'a expr) : 'a expr =
match a with
| Const(_, _) → Const(fin, _)
| Plus(x, a, b) → Plus(+, corrige_position a,
                        corrige_position b)
;;
```

**Question 10** (2 pt) Donner le type et le code d'une fonction `annotation_map` qui prend une fonction qui transforme une annotation en une autre et qui va utiliser cette fonction pour transformer toutes les annotations d'une expression, produisant ainsi une nouvelle expression annotée.

2/2

```
type : ('a -> 'b) -> 'a expr -> 'b expr
let rec annotation_map (f:'a->'b) (a:'a expr):
'b expr =
  match a with
  | Const (y1,y2) -> Const((f y1), y2)
  | Plus (x,x1,x2) -> Plus((f x), annotation_map f x1,
                           annotation_map f x2)
```

**Question 11** (2 pt) Donner le type et le code d'une fonction `annotation_combine` qui prend une fonction qui combine trois annotations (celle d'un nœud et deux annotations venant de ses fils) en une seule et qui va produire l'annotation combinée à partir des annotations d'une expression.

0/2

```
type annotation_combine :
let annotation_combine (a:'a expr) : 'a expr =
match a with
| Const (x, y) -> Const (x, y)
| Plus (a, Plus(b,-,-), Plus(c,-,-)) ->
    Plus((a,b,c)), annotation_combine Plus(b,-,-),
    annotation_combine Plus(c,-,-))
;;
```

+196/8/53+